Simulación realista de adversarios como herramienta formativa para DFIR y Blue Team

Introducción

- Escenario didáctico
- Aproximar al analista la mentalidad del atacante
- Simulación de adversarios
  - Evalúa sistemas de defensa
  - Evalúa capacidades del personal
  - **Formación del personal**
    - Detección, análisis e interpretación de un incidente completo

Índice

1. Enunciado
2. El incidente
3. Red Team notes
4. Conclusiones

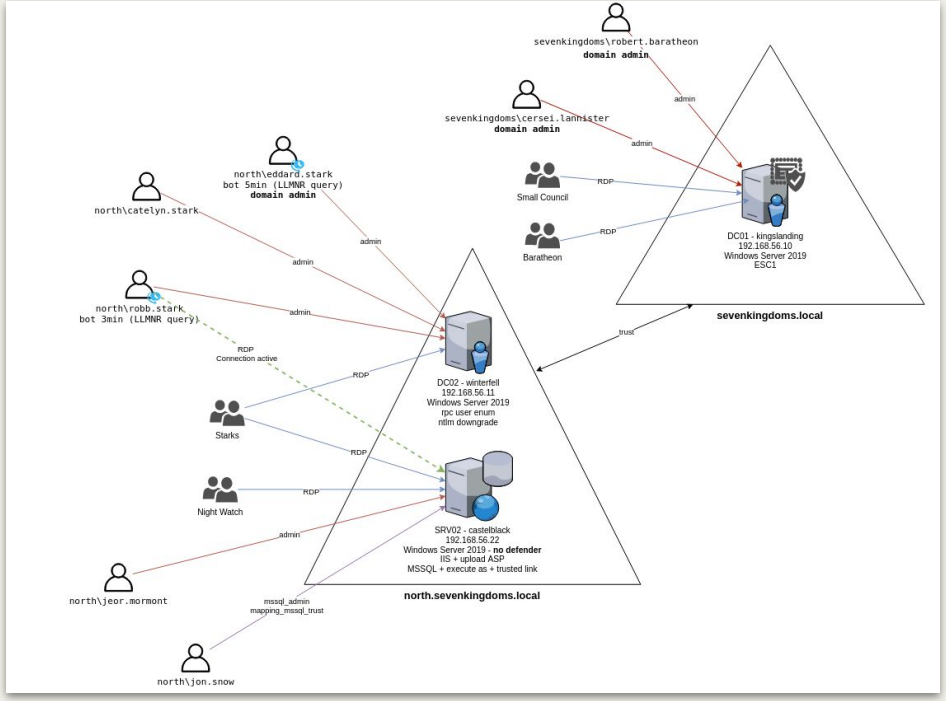
1

Escenario

La Victima

- Windows AD (GOAD-Light)
  - Dos dominios
  - Una Workstation Win 11

- Monitorizacion
  - Windows Event Logs
  - Sysmon
  - Suricata
  - Memory dump



Recopilación de evidencias

- Entrega de artefactos para Analistas
  - Google Drive (QR)

- Artefactos entregados
  - Volcado de la memoria RAM
  - KAPE
    - !SANS_Triage
    - EventLogs

2

El incidente

Resumen ejecutivo

- Acciones sobre el objetivo
- Mando y Control (C2)
- Instalación
- Explotación
- Entrega
- Armamento
- Reconocimien

3

Red Team notes

Sistema de Staging

- **Stage 1**
  - Reverse Shell
- **Stage 2**
  - DLL Hijacking
  - Side Loading
  - DLL Proxying
  - Persistencia

3.1

Entrega

BadUSB

- **Primer punto de interacción con la víctima**: inicio del incidente
- **Objetivo**: introducir el primer payload en el entorno
- **Procedimiento**:
  - Uso de dispositivo BadUSB como teclado (HID)
  - Inyección automática de pulsaciones de teclado
  - Apertura de ejecución (Win + R) y consola PowerShell
  - Descarga y ejecución de payload sin interacción del usuario

Bypass de AMSI

- Ejecución inicial mediante **PowerShell** (BadUSB)
- AMSI analiza scripts antes de ejecutarlos
- Primer mecanismo defensivo a evadir

Técnica 1: Memory Patching

- Modificación de AmsiScanBuffer() en memoria
- Fuerza resultado “limpio” (ej. E_INVALIDARG)
- Permite ejecución del código
- Puede ser detectado → finaliza el proceso

Técnica 2: Memory Patching via Reflection

- Manipulación de AmsiUtils.amsiInitFailed
- Se establece en true → AMSI se desactiva
- No requiere parcheo directo
- Más eficaz en entornos con detección activa

3.2
Explotación

Stage 1 (ReverseShell)

- **Primera ejecución de código en el sistema víctima**
- **Objetivo**: desplegar payload inicial y establecer comunicación con atacante
- **Procedimiento**:
  - Ejecución de script PowerShell contra infraestructura C2
  - Descarga de archivo temporal en `$env:TEMP`
  - Renombrado a `.exe` y ejecución oculta
  - Establecimiento de reverse shell para acceso remoto al sistema

Shellcode

- Código ensamblador position-independent
- Se ejecuta en cualquier dirección de memoria
- No depende de un entry point fijo
- Uso principal: explotación y ejecución arbitraria

Position-Independent Code (PIC)

- El shellcode debe ser **independiente de la dirección**
- No conoce su ubicación en memoria
- Necesita calcularla en tiempo de ejecución
- **Técnica clave: call-pop**
  - Recupera el valor de EIP
  - Permite calcular su propia dirección
  - Uso de direcciones relativas para datos/código

Runtime Linking (Concepto)

- Necesidad de llamar a APIs del sistema (ej. kernel32.dll)
- No hay linker ni loader disponibles
- Resolución de funciones en **tiempo de ejecución**

Runtime Linking (Proceso)

- Acceso al **PEB** (FS:[0x30] / GS:[0x60])
- Enumeración de módulos cargados
- Obtención de base address
- Análisis de la **Export Table**
  - Cálculo: func_addr = base + RVA
  - Permite invocar APIs dinámicamente

Hashing de funciones (Evasión)

- Evita strings en claro en el shellcode
- Uso de **hashing (XOR + rotaciones)**
- Comparación de hashes en runtime
- Dificulta análisis estático

ABI en x86 vs x64

- **x86**
  - Argumentos por pila (right-to-left)
- **x64**
  - Registros: RCX, RDX, R8, R9
  - Alineación de pila (16 bytes)
  - Shadow space obligatorio (32 bytes)
- Cumplimiento crítico → evitar fallos

Payloads finales

- Basados en: PIC + Runtime Linking + ABI
- **Metodología**
  - Prototipo en C
  - Análisis con herramientas (ej. IDA)
- **Ejemplos**
  - MessageBox (testing)
  - Reverse shell (payload final)

3.3
Instalación

Stage 2 (DLL Hijacking)

- **Despliegue del payload final tras obtener reverse shell**
- **Objetivo:** establecer acceso persistente en el sistema comprometido
- **Procedimiento**:
  - Uso de ejecutable legítimo (Calibre) en directorio con permisos de escritura
  - Control del entorno de carga de DLLs
  - Renombrado de DLL legítima y sustitución por DLL maliciosa (proxy)
  - Intercepción de llamadas y ejecución de código malicioso

Inyección de código

- Ejecución de código no previsto en un proceso
- Modificación del flujo de ejecución
- Inserción de código en memoria
- Técnicas comunes:
  - Inyección de shellcode
  - Carga forzada de DLLs
- Técnica clave: **DLL Hijacking**

DLL Hijacking (Concepto)

- Forzar carga de una DLL maliciosa
- Aprovecha el orden de búsqueda de Windows
- **Objetivo**
  - Ejecutable legítimo carga DLL controlada
  - Sustituye a una DLL esperada

Weaponización (Visión general)

- Técnicas combinadas:
  - Side Loading
  - DLL Proxying
  - *Reflective Loading*
- Objetivo:
  - Ejecutar código en proceso legítimo
  - Mantener operación **fileless**

Side Loading

- DLL maliciosa en **mismo directorio**
- **Mismo nombre** que la DLL legítima
- Requisito:
  - Aplicación vulnerable
- Solución a permisos:
  - Copiar ejecutable a ruta escribible
  - Insertar DLL maliciosa

DLL Proxying

- **Problema**
  - Sustituir DLL rompe funcionalidad
  - Faltan funciones originales
- **Solución**
  - DLL maliciosa:
    - Ejecuta código ofensivo
    - Reenvía funciones reales (Ficheros .def)
  - Mantiene funcionamiento normal
  - Oculta comportamiento malicioso

Ficheros .def

- Definen exportaciones de la DLL
- Permiten:
  - Replicar funciones
  - Redirigir a DLL real
- Ejemplo:
  - FuncB → real.FuncB
  - Conserva ordinales

DLL Hijacking && Side Loading && DLL Proxying

- Aplicación vulnerable en ruta no escribible
- Se copia a un directorio controlado (escribible)
- Se añade DLL maliciosa con el mismo nombre
- El ejecutable carga nuestra DLL (**Side Loading**)
- **DLL Proxying**
  - Reenvía funciones a la DLL original
  - Mantiene la funcionalidad del programa
- **Resultado**
  - Código malicioso en proceso legítimo
  - Ejecución transparente para el usuario

Persistencia

- **Mantenimiento del acceso remoto tras reinicios**
- **Objetivo:** mantener acceso persistente tras reinicios del sistema
- **Procedimiento**:
  - Creación de una tarea programada (`calibre-update`)
  - Configurada para ejecutarse en cada reinicio
  - Invocación de un binario legítimo (`calibre-debug.exe`) sin interfaz gráfica
  - El binario recarga la DLL maliciosa

Persistencia (Concepto)

- Mantener acceso tras comprometer el sistema
- Ejecución automática del payload
- Métodos comunes:
- Claves de registro
  - Servicios
  - Tareas programadas
  - Carpeta Start-up
  - DLL Hijacking

Tarea programada (schtasks)

- Creación de tarea: CalibreUpdater
- Frecuencia: diaria
- Simula actualización legítima
- **Ejecuta**:
  - Binario aparentemente legítimo
  - Payload malicioso oculto
- **Camuflaje**
  - Nombre creíble (actualización de software)
  - Ubicación coherente con la aplicación
  - Comportamiento similar a tareas reales

3.4
Mando y Control (C2)

Sliver C2

- **Objetivo:** control continuo del sistema comprometido mediante Sliver C2
- **Procedimiento**:
  - Carga del implante Sliver en memoria mediante DLL maliciosa
  - Ejecución mediante reflective loading (sin APIs estándar ni artefactos en disco)
  - Establecimiento de conexión con servidor C2
  - Comunicación mediante tráfico HTTPS cifrado
  - Uso de beaconing para intercambio de comandos y resultados

Reflective Loading (Concepto)

- Carga de una **DLL directamente en memoria**
- Evita uso de LoadLibrary
- No requiere archivo en disco
- Objetivo:
  - Ejecución **fileless**
  - Reducir detección

Proceso general

- DLL ya disponible como bytes en memoria
- Reconstrucción manual dentro del proceso
- Ejecución sin APIs estándar

Pasos - Reflective Loading

1. Procesar las cabeceras
2. Procesar las cabeceras
3. Reservar memoria para la DLL
4. Copiar las cabeceras
5. Copiar las cabeceras
6. Resolver la tabla de importaciones.
7. Aplicar relocaciones
8. Invocar DllMain

3.4

Acciones sobre el objetivo

Acciones finales (Alcance del trabajo)

- El trabajo implementa:
  - Infraestructura C2 con **Sliver**
  - Implant inicial y comunicación
- Esta sección describe:
  - Técnicas posibles tras la ejecución del implante
  - Métodos para ejecutar payloads adicionales en memoria
- No todas las técnicas se implementan completamente
- Se presentan como **opciones operativas** dentro de un entorno Red Team

Objetivo de estas técnicas

- Ejecutar código adicional sobre el sistema víctima
- Mantener ejecución en memoria (fileless)
- Extender capacidades del implante
- Evitar artefactos en disco

BOF / COFF

- Objetos compilados en formato .obj (no enlazados)
- Ejecutados directamente en memoria por el implante
- Código nativo sin necesidad de ejecutables completos
- **Características**:
  - No requieren linking ni loader tradicional
  - Dependencias mínimas
  - Formato compacto y portátil
- **Uso típico**:
  - Ejecución de funciones específicas
  - Operaciones post-explotación en memoria
  - Integración con frameworks ofensivos

Shared Library Side Loading

- Carga de DLLs en memoria sin usar el loader estándar de Windows
- Mapeo manual del PE en el espacio de direcciones del proceso
- **Componentes clave**:
  - Resolución manual de símbolos exportados
  - Localización de funciones dentro de la DLL
  - Ejecución controlada del código cargado
- **Capacidades**:
  - Suplantación de DLLs legítimas
  - Redirección del flujo de ejecución
  - Integración con técnicas de DLL Hijacking

Assemblies .NET

- Código compilado en **Intermediate Language (IL)**
- Ejecutado por el **Common Language Runtime (CLR)**
- Ventajas:
  - Alto nivel de abstracción
  - Acceso directo a APIs de .NET
  - Flexibilidad en el desarrollo de payloads
- Ejecución:
  - In-process: dentro del propio implante
  - Proceso sacrificado: en un proceso separado
- Requiere:
  - Carga dinámica del runtime mediante runtime hosting

4

Conclusiones

Gracias por su atención